Neural Compression for Power Spectral Density

Martín Ramírez Espinosa, Sergio Alejandro Gil Ríos

February 2026

1 Introduction

Wideband spectrum monitoring based on low-cost Software Defined Radios (SDRs) is a practical approach to assess occupancy in the FM broadcast band and to support regulatory and operational compliance tasks. In these systems, a sensing node continuously acquires complex baseband samples, computes spectral features (in particular, a Power Spectral Density (PSD) estimate), and reports results to a remote server for visualization and decision-making.

A key bottleneck is communication and storage: sending raw I/Q streams or high-resolution PSD frames at high cadence can easily exceed uplink capacity and backend storage/processing budgets. This work formulates PSD compression as a rate–distortion problem and proposes a VQ-VAE-based system model that learns a compact discrete representation (bitstream) of the PSD while preserving the quantities used for FM spectrum sensing, such as occupancy maps and emission-related features.

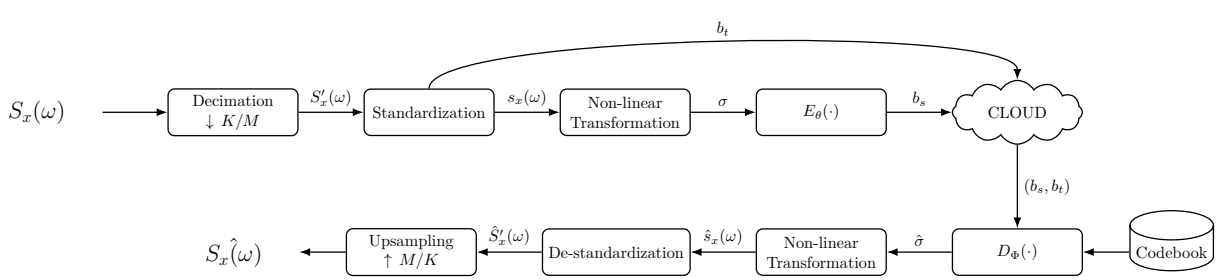
2 Problem Formulation

2.1 System model

We consider a sensing node that, at discrete time instants t, computes a PSD estimate of a received process $x(t)$ over a band of interest. Denote by

$$\mathbf{S}_t \triangleq \left[S_t(\omega_1), \ldots, S_t(\omega_N)\right]^\top \in \mathbb{R}_+^N \tag{1}$$

the discretized PSD sampled on a uniform frequency grid $\{\omega_n\}_{n=1}^N$ covering the monitored band. The node must transmit a compressed description of $\mathbf{S}_t$ to a cloud service under a bitrate budget, where the cloud reconstructs $\hat{\mathbf{S}}_t$ and performs spectrum-sensing analytics.



2.2 Sensing-driven quantities derived from the PSD

The remote monitor uses the reconstructed PSD to derive spectrum-sensing quantities relevant to FM monitoring. Typical examples include:

$$\omega_{\text{pk}} \triangleq \arg\max_{\omega} S_t(\omega) \qquad \text{(peak/candidate carrier)} \tag{2}$$

$$\omega_{\text{cent}} \triangleq \frac{\sum_{n=1}^{N} \omega_n S_t(\omega_n)}{\sum_{n=1}^{N} S_t(\omega_n)} \qquad \text{(spectral centroid).} \tag{3}$$

Additionally, an occupancy map can be obtained by comparing the PSD to a (possibly frequency-dependent) noise-floor estimate $N_b(\omega)$ plus a margin γ:

$$O_t(\omega_n) \triangleq \mathbf{1}\{S_t(\omega_n) > N_b(\omega_n) + \gamma\}. \tag{4}$$

Connected occupied regions define candidate emissions and may be used to estimate an occupied bandwidth B_{occ} (e.g., by thresholded support width or by a power-containment rule).

2.3 PSD compression pipeline (VQ-VAE system model)

For each PSD frame $\mathbf{S}_t$, the sensing node applies a deterministic pre-/post-processing chain and a learned VQ-VAE bottleneck.

Decimation (frequency downsampling). A deterministic operator reduces frequency resolution:

$$\mathbf{S}'_t = \mathcal{D}_{K/M}(\mathbf{S}_t) \in \mathbb{R}_+^{N'}, \qquad N' \approx N \cdot \frac{K}{M}. \tag{5}$$

Standardization and side information. We standardize the decimated PSD using parameters $\mathbf{t}_t$ (e.g., mean/scale, min/max, or blockwise gains),

$$\mathbf{s}_t = \text{Std}(\mathbf{S}'_t; \mathbf{t}_t), \tag{6}$$

and transmit a quantized representation of these parameters as side-information bits b_t so the cloud can invert the operation.

Non-linear transformation. To better match the dynamic range of PSD values to the quantizer and the neural model, we apply an invertible pointwise mapping

$$\boldsymbol{\sigma}_t = g_{\text{nl}}(\mathbf{s}_t), \qquad \mathbf{s}_t = g_{\text{nl}}^{-1}(\boldsymbol{\sigma}_t). \tag{7}$$

Examples include log-like compressions and other monotone mappings that reduce heavy tails.

VQ-VAE encoder, vector quantization, and bitstream b_s. The encoder produces latent features $\mathbf{z}_t = E_\theta(\boldsymbol{\sigma}_t)$ which are partitioned into blocks. Each block is quantized to the nearest codeword in a learned codebook $\mathcal{E} = \{\mathbf{e}_1, \ldots, \mathbf{e}_{|\mathcal{E}|}\}$:

$$\mathbf{k}_t = Q_{\mathcal{E}}(\mathbf{z}_t), \qquad \hat{\mathbf{z}}_t = \mathcal{E}[\mathbf{k}_t]. \tag{8}$$

The indices $\mathbf{k}_t$ are entropy-coded into a primary bitstream b_s (main payload). The total transmitted/stored payload per PSD frame is therefore (b_s, b_t).

Decoder and inverse processing. At the cloud, the decoder reconstructs

$$\hat{\boldsymbol{\sigma}}_t = D_\Phi(\hat{\mathbf{z}}_t), \tag{9}$$

$$\hat{\mathbf{s}}_t = g_{\mathrm{nl}}^{-1}(\hat{\boldsymbol{\sigma}}_t), \tag{10}$$

$$\hat{\mathbf{S}}'_t = \mathrm{DeStd}(\hat{\mathbf{s}}_t; \mathbf{t}_t), \tag{11}$$

$$\hat{\mathbf{S}}_t = \mathcal{U}_{M/K}(\hat{\mathbf{S}}'_t), \tag{12}$$

where $\mathcal{U}_{M/K}$ is an upsampling/interpolation operator restoring the original frequency grid.

2.4 Bit budget and rate–distortion objective

Let $L_{b_s}(t)$ and $L_{b_t}(t)$ denote the number of bits used by the main bitstream and the side information, respectively. The total payload per PSD frame is

$$L_b(t) = L_{b_s}(t) + L_{b_t}(t). \tag{13}$$

We seek to compress the PSD while preserving both waveform-level fidelity and sensing-level decisions derived from the PSD.

A rate–distortion formulation is:

$$\min_{\theta,\Phi,\mathcal{E}} \ \mathbb{E}\big[\mathcal{D}(\mathbf{S}_t, \hat{\mathbf{S}}_t)\big] \quad \text{s.t.} \quad \mathbb{E}[L_b(t)] \le R_{\max}, \tag{14}$$

or equivalently the Lagrangian form:

$$\min_{\theta,\Phi,\mathcal{E}} \ \mathbb{E}\Big[\lambda\, \mathcal{D}(\mathbf{S}_t, \hat{\mathbf{S}}_t) + L_b(t)\Big], \tag{15}$$

where $\lambda > 0$ controls the rate–distortion trade-off.

2.5 Distortion tailored to spectrum sensing

Because the cloud ultimately performs sensing on $\hat{\mathbf{S}}_t$, we define $\mathcal{D}$ to penalize errors that matter for monitoring:

1. **PSD fidelity.** A normalized reconstruction error (often in log-PSD) such as

$$\mathcal{D}_{\mathrm{psd}} = \frac{1}{N} \sum_{n=1}^{N} \big(\psi(S_t(\omega_n)) - \psi(\hat{S}_t(\omega_n))\big)^2, \tag{16}$$

   with $\psi(\cdot)$ an optional dynamic-range map (e.g., $\log(\epsilon + \cdot)$).

2. **Occupancy consistency.** Let $O_t(\omega_n)$ and $\hat{O}_t(\omega_n)$ be computed from S_t and $\hat{S}_t$ using the same noise-floor rule. Penalize mismatches, e.g., by a weighted binary cross-entropy (or FP/FN-weighted) loss across frequency bins.

3. **Feature preservation.** Penalize deviations in sensing features such as peak and centroid:

$$\mathcal{D}_{\mathrm{feat}} = \alpha_{\mathrm{pk}} |\omega_{\mathrm{pk}} - \hat{\omega}_{\mathrm{pk}}| + \alpha_{\mathrm{cent}} |\omega_{\mathrm{cent}} - \hat{\omega}_{\mathrm{cent}}| + \alpha_B |B_{\mathrm{occ}} - \hat{B}_{\mathrm{occ}}|. \tag{17}$$

A practical overall distortion is a weighted sum,

$$\mathcal{D} = \beta_{\mathrm{psd}} \mathcal{D}_{\mathrm{psd}} + \beta_{\mathrm{occ}} \mathcal{D}_{\mathrm{occ}} + \beta_{\mathrm{feat}} \mathcal{D}_{\mathrm{feat}}. \tag{18}$$

2.6 VQ-VAE training losses (codebook and commitment)

In addition to the rate–distortion objective above, VQ-VAE training typically includes terms that stabilize the discrete bottleneck. Let $\mathrm{sg}[\cdot]$ denote the stop-gradient operator. A common choice is:

$$\mathcal{L}_{\mathrm{VQ}} = \left\|\mathrm{sg}[\mathbf{z}_t] - \hat{\mathbf{z}}_t\right\|_2^2 + \beta_{\mathrm{com}}\left\|\mathbf{z}_t - \mathrm{sg}[\hat{\mathbf{z}}_t]\right\|_2^2, \tag{19}$$

where the first term updates the codebook and the second (commitment) term encourages encoder outputs to stay close to selected codewords.

Overall training problem. Combining the sensing-aware distortion, the bit budget, and the VQ terms yields the final learning problem:

$$\min_{\theta,\Phi,\mathcal{E}} \; \mathbb{E}\Big[\lambda\,\mathcal{D}(\mathbf{S}_t, \hat{\mathbf{S}}_t) + L_b(t)\Big] + \eta\,\mathbb{E}\big[\mathcal{L}_{\mathrm{VQ}}\big], \tag{20}$$

with hyperparameters $\lambda, \eta, \beta_{\mathrm{com}}$ and the sensing-aware weights inside $\mathcal{D}$.